

Tháng 3 2026

FLYWEIGHT

Trình bày bởi Nhóm 5

NỘI DUNG

01. Vấn đề cần giải quyết

02. Định nghĩa flyweight

03. Ý tưởng cốt lõi

04. Thành phần chính

05. UML

06. Luồng hoạt động

07. Đánh giá

08. So sánh

VẤN ĐỀ CẦN GIẢI QUYẾT

Đặt tình huống thực tế:

- Trong một số hệ thống, ta cần tạo ra rất nhiều object giống nhau hoặc gần giống nhau.

Ví dụ:

- Game có hàng ngàn viên đạn, cây cối, quân lính.
- Trình soạn thảo văn bản có hàng triệu ký tự.
- Bản đồ có rất nhiều icon: nhà hàng, cây xăng, khách sạn.
- App giao hàng hiển thị rất nhiều marker shipper/order trên bản đồ.
- Hệ thống vẽ đồ họa có rất nhiều hình tròn, hình vuông, particle.

ĐỊNH NGHĨA FLYWEIGHT PATTERN

Flyweight Pattern là một design pattern thuộc nhóm Structural Pattern.

Nó giúp giảm lượng bộ nhớ sử dụng bằng cách chia sẻ những phần trạng thái giống nhau giữa nhiều object thay vì lưu trữ lặp lại trong từng object.

Nói đơn giản:

- Flyweight là pattern dùng để tái sử dụng object nhỏ, nhẹ, dùng chung cho nhiều context khác nhau.

Ý TƯỞNG CỐT LÕI

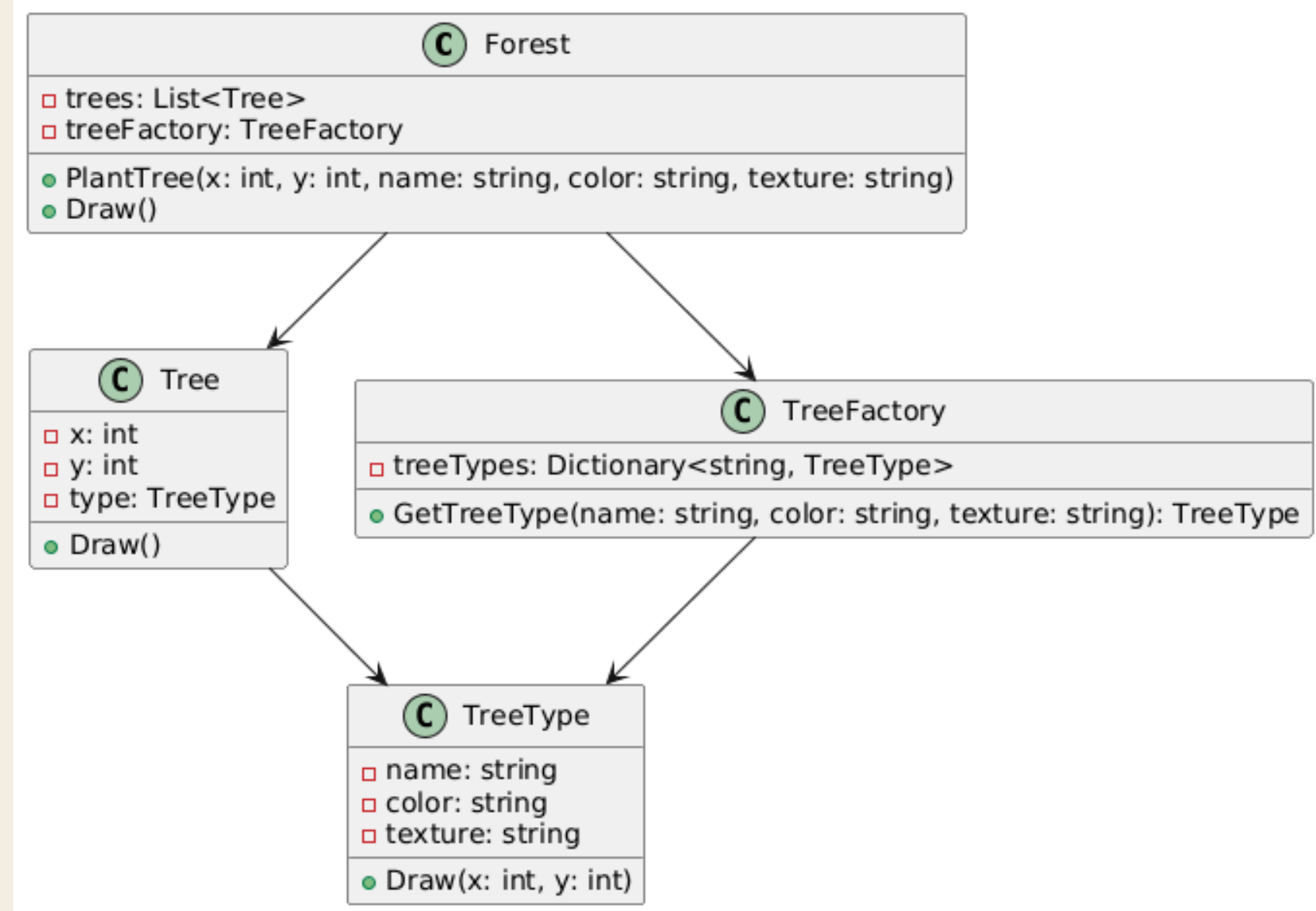
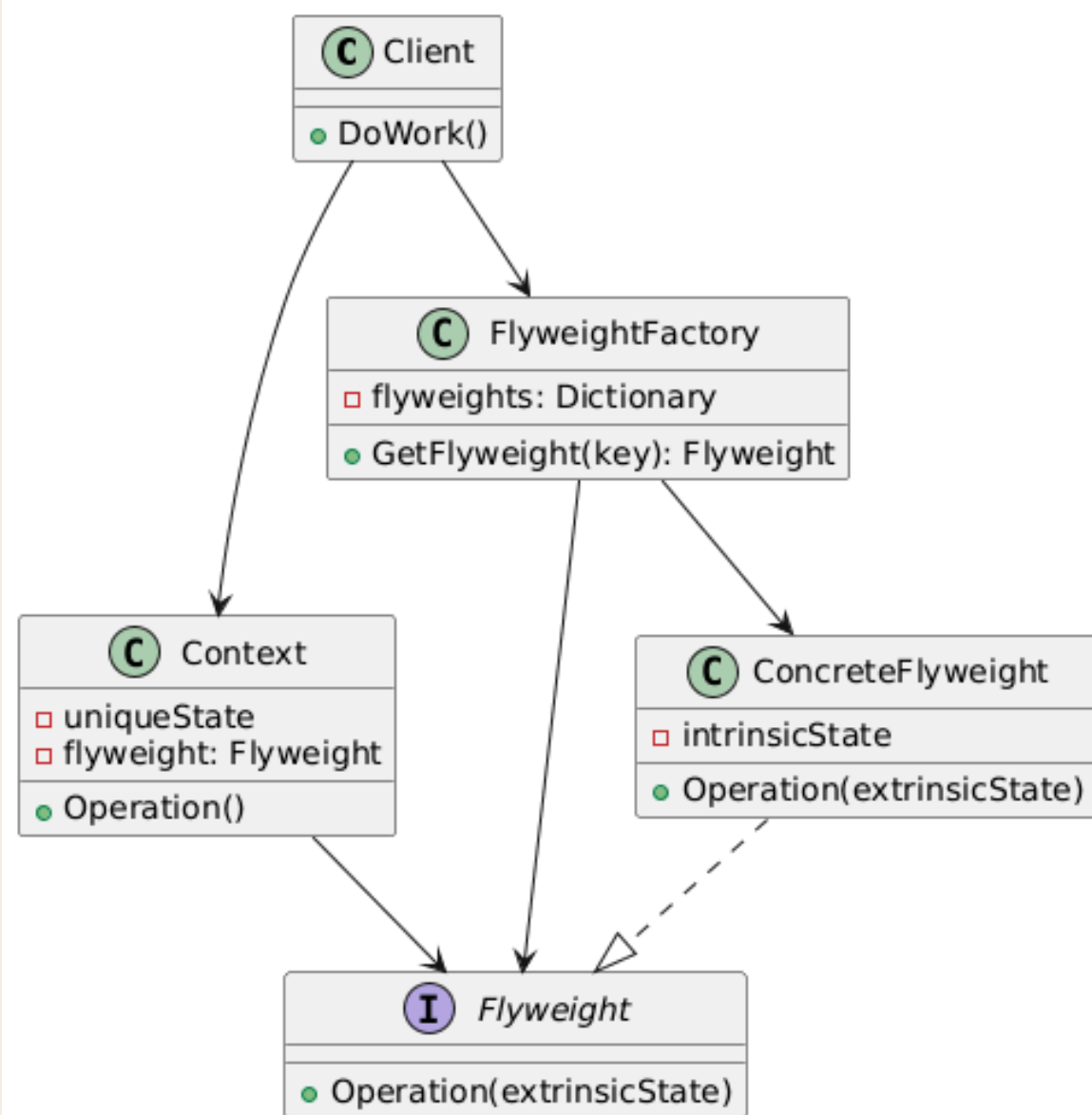
Ý tưởng chính của Flyweight Pattern là:

- Không lưu dữ liệu giống nhau trong từng object. Hãy lưu dữ liệu giống nhau vào một object dùng chung, còn dữ liệu riêng thì truyền từ bên ngoài hoặc lưu riêng.

THÀNH PHẦN CHÍNH

Thành phần	Vai trò
Flyweight	Interface hoặc class chứa phần trạng thái có thể chia sẻ
ConcreteFlyweight	Object thật sự được chia sẻ
FlyweightFactory	Quản lý, tái sử dụng hoặc tạo mới Flyweight
Context	Object chứa trạng thái riêng, tham chiếu đến Flyweight
Client	Sử dụng Context hoặc yêu cầu Factory cung cấp Flyweight

UML



LUỒNG HOẠT ĐỘNG

Luồng hoạt động của Flyweight Pattern:

1. Client cần tạo một object mới.
2. Client gửi thông tin định danh phần dùng chung cho FlyweightFactory.
3. Factory kiểm tra xem Flyweight tương ứng đã tồn tại chưa.
4. Nếu đã tồn tại, Factory trả về object cũ.
5. Nếu chưa tồn tại, Factory tạo object mới, lưu lại rồi trả về.
6. Context/object riêng lưu phần trạng thái riêng và tham chiếu đến Flyweight.
7. Khi cần xử lý, Context gọi Flyweight và truyền thêm trạng thái riêng nếu cần.

ĐÁNH GIÁ

Ưu điểm	Nhược điểm
Tiết kiệm bộ nhớ	Tăng độ phức tạp thiết kế
Giảm số lượng object nặng	Khó debug hơn
Tăng hiệu quả khi xử lý số lượng lớn object	Không hiệu quả nếu object không có nhiều dữ liệu lặp lại
Tách rõ dữ liệu dùng chung và dữ liệu riêng	Có thể tốn thêm chi phí lookup

SO SÁNH

Flyweight vs Singleton

Tiêu chí	Flyweight	Singleton
Mục đích chính	Chia sẻ nhiều object nhỏ có trạng thái giống nhau	Đảm bảo một class chỉ có một instance
Số lượng instance	Có thể có nhiều instance, mỗi instance đại diện một loại state dùng chung	Chỉ một instance duy nhất
Trọng tâm	Tiết kiệm bộ nhớ	Kiểm soát số lượng instance
Có Factory/cache không?	Thường có	Không bắt buộc
Ví dụ	Nhiều TreeType: Oak, Pine, Cherry	Một ConfigurationManager

SO SÁNH

Flyweight vs Prototype

Tiêu chí	Flyweight	Prototype
Mục đích chính	Chia sẻ object để giảm bộ nhớ	Clone object để tạo object mới nhanh hơn
Object được tạo ra	Dùng lại object có sẵn	Tạo bản sao mới
Trạng thái	Tách intrinsic/extrinsic state	Mỗi clone thường có state riêng
Có chia sẻ object không?	Có	Không nhất thiết
Ví dụ	Dùng chung TreeType	Clone một Enemy mẫu thành enemy mới

SO SÁNH

Flyweight vs Object Pool

Tiêu chí	Flyweight	Object Pool
Mục đích chính	Chia sẻ dữ liệu giống nhau để tiết kiệm bộ nhớ	Tái sử dụng object tạm thời để tránh tạo/hủy nhiều lần
Object có dùng đồng thời không?	Có thể được nhiều context dùng cùng lúc	Thường một object được mượn bởi một client tại một thời điểm
State	Flyweight thường chứa state dùng chung, ít thay đổi	Object trong pool thường có state thay đổi và cần reset
Ví dụ	Dùng chung TreeType cho nhiều cây	Pool connection database, bullet object trong game

Tháng 3 2026

THANK YOU

for Listening

Trình bày bởi Nhóm 5